

GUILDBOARD

Documentation technique - Backend

Spring Boot + PostgreSQL + JPA

Projet : GuildBoard

Backend : Spring Boot 4.0.8

Java : 26

Base de donnees : PostgreSQL 18.6

API REST : /api/aventuriers, /api/quetes, /api/assignments

Sommaire

- 1. Objet et perimetre du backend
- 2. Technologies et versions
- 3. Arborescence du projet
- 4. Architecture generale
- 5. Configuration Spring Boot
- 6. Base de donnees PostgreSQL
- 7. Entites JPA
- 8. DTO et validation
- 9. Mappers
- 10. Repositories
- 11. Services
- 12. Controllers et API REST
- 13. Fonctionnement d une requete
- 14. Tests automatises
- 15. Tests manuels avec Bruno
- 16. Commandes de lancement et maintenance
- 17. Git et livraison du backend
- 18. Points techniques a connaitre pour la soutenance
- 19. Glossaire

1. Objet et perimetre du backend

Le backend de GuildBoard fournit une API REST permettant de gerer les donnees principales de l'application : les aventuriers, les quetes et les assignations entre un aventurier et une quete. Il assure la validation des donnees, la logique metier, l'accès à PostgreSQL et la transformation des objets envoyés ou reçus par le frontend.

Le backend est organisé en couches. Cette separation permet de garder le code lisible et de donner à chaque couche une responsabilité précise.

Couche	Responsabilite
Controller	Reçoit les requetes HTTP et retourne les reponses HTTP.
DTO	Transporte les donnees entre le client et le backend et porte les validations.
Service	Contient la logique metier et orchestre les operations.
Mapper	Convertit Entity <-> DTO.
Repository	Communique avec PostgreSQL via Spring Data JPA.
Entity	Représente les tables de la base de donnees.

Ce que le backend gere

- CRUD complet des aventuriers : GET, POST, PUT, DELETE.
- CRUD complet des quetes : GET, POST, PUT, DELETE.
- CRUD complet des assignations : GET, POST, PUT, DELETE.
- Validation des champs avec Jakarta Validation.
- Relations JPA entre Aventurier, Assignation et Quete.
- Connexion à PostgreSQL avec Hibernate/JPA.

2. Technologies et versions

Technologie	Version / usage
Java	26
Spring Boot	4.0.8
Spring Data JPA	Fourni via spring-boot-starter-data-jpa
Spring Web MVC	Fourni via spring-boot-starter-webmvc
Jakarta Validation	spring-boot-starter-validation
PostgreSQL	18.6
Driver PostgreSQL	org.postgresql:postgresql
Maven	3.9.16
Tests	JUnit 5 + Spring Boot Test + TestRestTemplate
Tests API manuels	Bruno
Frontend associe	React + TypeScript + Vite

Le backend est autonome : le frontend consomme son API REST sur le port 8080 lorsque l'application Spring Boot est lancée.

3. Arborescence du projet

```
C:\Projet Guildboard\
+-- frontend\
```

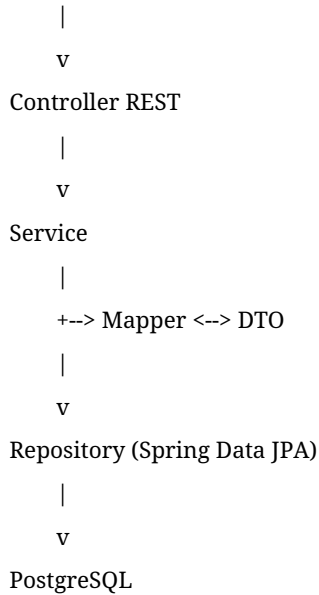
```

+-- backend\
+-- pom.xml
+-- mvnw
+-- mvnw.cmd
+-- src\
+-- main\
| +-- java\com\example\backend\
| | +-- BackendApplication.java
| | +-- controller\
| | | +-- AventurierController.java
| | | +-- QueteController.java
| | | +-- AssignmentController.java
| | +-- dto\
| | | +-- AventurierDTO.java
| | | +-- QueteDTO.java
| | | +-- AssignmentDTO.java
| | +-- entity\
| | | +-- Aventurier.java
| | | +-- Quete.java
| | | +-- Assignment.java
| | +-- mapper\
| | | +-- AventurierMapper.java
| | | +-- AssignmentMapper.java
| | +-- repository\
| | | +-- AventurierRepository.java
| | | +-- QueteRepository.java
| | | +-- AssignmentRepository.java
| | +-- service\
| | | +-- AventurierService.java
| | | +-- QueteService.java
| | | +-- AssignmentService.java
| +-- resources\
|   +-- application.properties
+-- test\
+-- java\com\example\backend\controller\
+-- AventurierControllerTest.java
+-- QueteControllerTest.java
+-- AssignmentControllerTest.java

```

4. Architecture generale

Client / Frontend



Le controller ne doit pas contenir la logique métier principale. Il délègue au service. Le service utilise les repositories pour accéder aux données et les mappers pour transformer les entités en DTO lorsque cela est nécessaire.

5. Configuration Spring Boot

5.1 pom.xml

```

<java.version>26</java.version>

<dependency>
  spring-boot-starter-data-jpa
</dependency>
<dependency>
  spring-boot-starter-webmvc
</dependency>
<dependency>
  spring-boot-starter-validation
</dependency>
<dependency>
  postgresql
  scope = runtime
</dependency>

```

Le pom.xml contient aussi les dépendances de test Spring Boot utilisées par la suite de tests.

5.2 application.properties

```
spring.application.name=backend
```

```

spring.datasource.url=jdbc:postgresql://localhost:5432/guildboard
spring.datasource.username=postgres
spring.datasource.password=<mot de passe local>
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

```

Propriete	Role
spring.datasource.url	Adresse de PostgreSQL et nom de la base guildboard.
spring.datasource.username	Utilisateur PostgreSQL utilise par Spring.
spring.datasource.password	Mot de passe local, non documente en clair ici.
ddl-auto=validate	Hibernate verifie que le schema existant correspond aux entites.
show-sql=true	Affiche les requetes SQL generees.
format_sql=true	Formate les requetes SQL pour les rendre plus lisibles.

6. Base de donnees PostgreSQL

La base de donnees du projet s'appelle guildboard. Le schema final utilise uniquement trois tables : aventurier, quete et assignation. Les anciennes tables guilds et users ont ete supprimees.

6.1 Table aventurier

Colonne	Type	Contraintes
id	INTEGER	PK, generation automatique
nom	VARCHAR(50)	NOT NULL, UNIQUE
classe	VARCHAR(50)	NOT NULL
niveau	INTEGER	NOT NULL, >= 1
xp	INTEGER	NOT NULL, >= 0
or	INTEGER	NOT NULL, >= 0

6.2 Table quete

Colonne	Type	Contraintes
id	INTEGER	PK, identity
titre	VARCHAR(100)	NOT NULL, UNIQUE
difficulte	VARCHAR(20)	NOT NULL
description	VARCHAR(500)	NOT NULL
niveau_requis	INTEGER	NOT NULL, >= 1
statut	VARCHAR(20)	NOT NULL
recompense_or	INTEGER	NOT NULL, >= 0
recompense_xp	INTEGER	NOT NULL, > 0

6.3 Table assignation

Colonne	Type	Contraintes
id	INTEGER	PK, identity
date_assignee	TIMESTAMP	NOT NULL
date_terminee	TIMESTAMP	nullable
aventurier_id	INTEGER	FK vers aventurier.id, NOT NULL
quete_id	INTEGER	FK vers quete.id, UNIQUE, NOT NULL

6.4 Relations

Aventurier 1 --- 0..* Assignment

Assignment 1 --- 1 Quete

Quete 0..1 --- 1 Assignment

Le champ quete_id est UNIQUE dans assignment. Cela signifie qu'une même quete ne peut pas être affectée à deux assignments simultanément.

Le champ SQL or est protégé dans le mapping JPA avec `@Column(name = "\"or\"")` pour conserver le nom de colonne tout en évitant les problèmes liés au mot SQL.

6.5 Commandes PostgreSQL utiles

```
C:\PostgreSQL\bin\psql.exe -U postgres -d guildboard
```

```
\l
```

```
\c guildboard
```

```
\dt
```

```
\d aventurier
```

```
\d quete
```

```
\d assignment
```

```
\q
```

7. Entites JPA

7.1 Aventurier

Element	Role
@Entity	Declare la classe comme entite JPA.
@Table(name="aventurier")	Associe la classe a la table PostgreSQL.
@Id	Declare la cle primaire.
@GeneratedValue(IDENTITY)	Demande a la base de generer l'identifiant.
@Column	Definit les contraintes de colonne.

Champs : id, nom, classe, niveau, xp, or. L'entité contient un constructeur vide JPA et les getters/setters nécessaires.

7.2 Quete

Champs : id, titre, difficulté, description, niveauRequis, statut, recompenseOr, recompenseXp. Les noms Java niveauRequis, recompenseOr et recompenseXp sont reliés aux noms SQL correspondants avec `@Column` lorsque nécessaire.

7.3 Assignment

Element	Role
dateAssignee	Date de debut de l'assignment.
dateTerminee	Date de fin ; peut rester NULL.

@ManyToOne aventurier	Plusieurs assignations peuvent concerner le meme aventurier.
@OneToOne quete	Une quete ne peut etre liee qu a une seule assignation.
@PrePersist	Initialise dateAssignee avec LocalDateTime.now() si elle est vide.

8. DTO et validation

Les DTO transportent les donnees de l API. Ils permettent aussi de valider les donnees entrantes sans exposer directement la structure interne des entites.

8.1 AventurierDTO

Champ	Validation
nom	@NotBlank, @Size(max=50)
classe	@NotBlank, @Size(max=50)
niveau	@NotNull, @Min(1)
xp	@NotNull, @Min(0)
or	@NotNull, @Min(0)

8.2 QueteDTO

Champ	Validation
titre	@NotBlank, @Size(max=100)
difficulte	@NotBlank, @Size(max=20)
description	@NotBlank, @Size(max=500)
niveauRequis	@NotNull, @Min(1)
statut	@NotBlank, @Size(max=20)
recompenseOr	@NotNull, @Min(0)
recompenseXp	@NotNull, @Min(1)

8.3 AssignationDTO

Champ	Validation
aventurierId	@NotNull, @Min(1)
queteId	@NotNull, @Min(1)
dateAssignee	LocalDateTime
dateTerminee	LocalDateTime

Dans les controllers, l annotation @Valid declenche la validation des DTO recus dans le corps JSON.

9. Mappers

9.1 AventurierMapper

AventurierMapper fournit deux conversions : toDTO(Aventurier) et toEntity(AventurierDTO).

- Entity -> DTO : copie id, nom, classe, niveau, xp et or.
- DTO -> Entity : reconstruit l entite a partir des valeurs du DTO.

9.2 AssignmentMapper

AssignmentMapper fournit toDTO(Assignation). Il copie les dates et transforme les objets Aventurier et Quete en leurs identifiants aventurierId et queteId.

Ce fonctionnement evite d envoyer directement toute la structure des entites associees dans la reponse JSON.

10. Repositories

Les repositories utilisent Spring Data JPA. Ils etendent JpaRepository et recuperent ainsi les operations CRUD de base sans ecrire de SQL pour les operations standards.

```
public interface AventurierRepository extends JpaRepository<Aventurier, Integer> {}
public interface QueteRepository extends JpaRepository<Quete, Integer> {}
public interface AssignmentRepository extends JpaRepository<Assignment, Integer> {}
```

Repository	Entite	Identifiant
AventurierRepository	Aventurier	Integer
QueteRepository	Quete	Integer
AssignmentRepository	Assignment	Integer

11. Services

Les services constituent la couche metier entre les controllers et les repositories.

11.1 AventurierService

- getAllAventuriers()
- getAventurierById(Integer id)
- createAventurier(AventurierDTO dto)
- updateAventurier(AventurierDTO dto)
- deleteAventurier(Integer id)

11.2 QueteService

- getAllQuetes()
- getQueteById(Integer id)
- createQuete(QueteDTO dto)
- updateQuete(QueteDTO dto)
- deleteQuete(Integer id)

11.3 AssignmentService

- getAllAssignations()
- getAssignmentById(Integer id)
- createAssignment(AssignmentDTO dto)
- updateAssignment(Integer id, AssignmentDTO dto)
- deleteAssignment(Integer id)

Lors de la creation ou de la mise a jour d une assignation, le service charge l Aventurier et la Quete correspondants puis les associe a l entite Assignation avant la sauvegarde.

12. Controllers et API REST

Les trois controllers exposent les ressources principales sous /api.

12.1 Aventuriers

Methode	Endpoint	Action	Reponse principale
GET	/api/aventuriers	Liste	200 OK
GET	/api/aventuriers/{id}	Detail	200 OK / 404
POST	/api/aventuriers	Creation	Objet cree
PUT	/api/aventuriers/{id}	Modification	200 OK / 404
DELETE	/api/aventuriers/{id}	Suppression	204 / 404

12.2 Quetes

Methode	Endpoint	Action	Reponse principale
GET	/api/quetes	Liste	200 OK
GET	/api/quetes/{id}	Detail	200 OK / 404
POST	/api/quetes	Creation	Objet cree
PUT	/api/quetes/{id}	Modification	200 OK / 404
DELETE	/api/quetes/{id}	Suppression	204 / 404

12.3 Assignations

Methode	Endpoint	Action	Reponse principale
GET	/api/assignations	Liste	200 OK
GET	/api/assignations/{id}	Detail	200 OK / 404
POST	/api/assignations	Creation	Objet cree
PUT	/api/assignations/{id}	Modification	200 OK / 404
DELETE	/api/assignations/{id}	Suppression	204 / 404

12.4 Annotations principales

Annotation	Role
@RestController	Indique que la classe expose une API REST.
@RequestMapping	Definit le prefixe de l URL.
@GetMapping	Expose une route GET.
@PostMapping	Expose une route POST.
@PutMapping	Expose une route PUT.
@DeleteMapping	Expose une route DELETE.
@RequestBody	Transforme le JSON du corps en objet Java.
@PathVariable	Recupere une variable presente dans l URL.
@Valid	Declenche les validations du DTO.
ResponseEntity	Permet de controler le corps et le code HTTP retournees.

13. Fonctionnement d une requete

Exemple : creation d une assignation.

1. Bruno ou le frontend envoie POST /api/assignations avec aventurierId et queteId.

2. AssignmentController recoit le JSON et le transforme en AssignmentDTO.
3. @Valid verifie que aventurierId et queteId sont fournis et >= 1.
4. AssignmentService charge l'Aventurier et la Quete via leurs repositories.
5. Le service cree ou met a jour l'entite Assignment.
6. @PrePersist complete automatiquement dateAssignee si necessaire.
7. AssignmentRepository sauvegarde l'entite dans PostgreSQL.
8. AssignmentMapper transforme l'entite en AssignmentDTO pour la reponse.
9. Le controller retourne le JSON avec le resultat de l'operation.

Exemple de requete POST

POST http://localhost:8080/api/assignments

Content-Type: application/json

```
{
  "aventurierId": 7,
  "queteId": 10
}
```

Exemple de reponse

```
{
  "aventurierId": 7,
  "dateAssignee": "2026-09-16T11:10:18.8614893",
  "dateTerminee": null,
  "id": 5,
  "queteId": 10
}
```

14. Tests automatisés

Les tests d'integration lancent l'application Spring Boot avec un port aleatoire et utilisent TestRestTemplate pour appeler les endpoints comme le ferait un client HTTP.

14.1 Classes de tests

- AventurierControllerTest
- QueteControllerTest
- AssignmentControllerTest

14.2 Import important avec Spring Boot 4

```
import org.springframework.boot.testclient.TestRestTemplate;
```

Le projet utilise l'import TestRestTemplate fourni avec la pile de tests Spring Boot 4 utilisee ici.

14.3 Commandes

```
cd "C:\Projet Guildboard\backend"  
.\mvnw.cmd -Dtest=AventurierControllerTest test  
.\mvnw.cmd -Dtest=QueteControllerTest test  
.\mvnw.cmd -Dtest=AssignmentControllerTest test  
.\mvnw.cmd clean test
```

La suite complete a ete executee avec succes et le build Maven a termine par BUILD SUCCESS.

15. Tests manuels avec Bruno

Les trois ressources ont ete testees manuellement avec Bruno. Les operations CRUD ont ete validees pour Aventurier, Quete et Assignment.

15.1 Exemple Assignment - POST

```
{  
  "aventurierId": 7,  
  "queteId": 10  
}
```

15.2 Exemple Assignment - PUT

```
{  
  "aventurierId": 7,  
  "queteId": 10,  
  "dateTerminee": "2026-09-16T11:30:00"  
}
```

La reponse retournee contenait bien dateTerminee apres modification, et la suppression retournait HTTP 204.

16. Commandes de lancement et maintenance

16.1 Lancer le backend

```
cd "C:\Projet Guildboard\backend"  
.\mvnw.cmd spring-boot:run
```

Par default, le backend est accessible sur <http://localhost:8080>.

16.2 Compiler sans lancer

```
.\mvnw.cmd clean test
```

16.3 Verifier Java et Maven

```
java -version
```

```
mvn -version
```

16.4 Demarrer PostgreSQL

Le service Windows PostgreSQL utilise dans le projet est postgresql-x64-18.

16.5 Connexion a PostgreSQL

```
C:\PostgreSQL\bin\psql.exe -U postgres -d guildboard
```

17. Git et livraison du backend

Le travail backend a ete versionne sur la branche feat/backend.

Element	Valeur
Branche	feat/backend
Commit principal	953bcaf
Message	Finalisation du backend et ajout des tests CRUD
Remote	https://github.com/amelbounneche-05/Projet-Guildboard.git

Le commit et le push vers la branche distante ont ete realises avec succes au moment de la finalisation du backend documente ici.

18. Points techniques a connaitre pour la soutenance

Architecture

- Controller = API HTTP.
- Service = logique metier.
- Repository = acces aux donnees.
- Entity = representation de la base.
- DTO = donnees envoyees/recues par l API.
- Mapper = conversion Entity <-> DTO.

JPA / Hibernate

- @Entity mappe une classe vers une table.
- @Id indique la cle primaire.
- @GeneratedValue genere l identifiant.
- @ManyToOne et @OneToOne representent les relations.
- @JoinColumn configure les cle etrangeres.

API REST

- GET lit les donnees.
- POST cree une ressource.

- PUT modifie une ressource.
- DELETE supprime une ressource.
- 200 = succes, 204 = succes sans contenu, 404 = ressource absente.

Validation

- @NotBlank : chaine obligatoire et non vide.
- @NotNull : valeur obligatoire.
- @Min : valeur numerique minimale.
- @Size : longueur maximale.

Questions probables

Question	Reponse courte
Pourquoi des DTO ?	Pour controler les donnees de l API et ne pas exposer directement les entites.
Pourquoi un Service ?	Pour separer la logique metier du controller.
Pourquoi un Repository ?	Pour acceder a PostgreSQL via Spring Data JPA sans ecrire tout le SQL CRUD.
Pourquoi @Valid ?	Pour appliquer automatiquement les contraintes du DTO aux donnees recues.
Pourquoi quete_id est UNIQUE ?	Pour garantir qu une quete ne puisse etre assignee qu une seule fois.
Pourquoi @PrePersist ?	Pour initialiser automatiquement dateAssignee avant l insertion.
Pourquoi ResponseEntity ?	Pour choisir explicitement le statut HTTP retourne.

19. Glossaire

Terme	Definition
API	Interface permettant a deux applications de communiquer.
REST	Style d API base sur HTTP et les ressources.
CRUD	Create, Read, Update, Delete.
DTO	Objet de transfert de donnees entre couches ou applications.
Entity	Objet Java lie a une table de la base.
JPA	API Java de persistance des donnees relationnelles.
Hibernate	Implementation JPA utilisee par Spring Boot.
Repository	Objet d acces aux donnees.
Mapper	Objet de conversion entre deux representations.
PK	Primary Key, identifiant unique d une ligne.
FK	Foreign Key, cle etrangere vers une autre table.
Validation	Controle automatique des donnees entrantes.

Annexe - Extraits importants du code

A. Repository

```
public interface AventurierRepository extends JpaRepository<Aventurier, Integer> {}
public interface QueteRepository extends JpaRepository<Quete, Integer> {}
public interface AssignmentRepository extends JpaRepository<Assignment, Integer> {}
```

B. Relation Assignment

```
@ManyToOne
@JoinColumn(name = "aventurier_id", nullable = false)
private Aventurier aventurier;

@OneToOne
@JoinColumn(name = "quete_id", nullable = false, unique = true)
private Quete quete;
```

C. Initialisation automatique de la date

```
@PrePersist
public void prePersist() {
    if (dateAssignee == null) {
        dateAssignee = LocalDateTime.now();
    }
}
```

D. Validation d un DTO

```
@NotBlank
@Size(max = 50)
private String nom;

@NotNull
@Min(1)
private Integer niveau;
```

Document base sur la structure et le code backend fournis pendant le developpement de GuildBoard. Les mots de passe locaux ne sont pas reproduits dans ce document.